Imagina

Clean Architecture para Frontend con React y Node

Joan Cifo

Sesión 1 · Martes 10 de noviembre de 2026 · 17:00 – 20:00

Índice de la sesión

5 MIN

De una aplicación que funciona a un diagnóstico priorizado con evidencias

Hora	Bloque	Contenido
17:00	**Apertura**	Curso, reglas, entorno y termómetro inicial
17:15	**Tema 1**	Introducción a Clean Architecture aplicada a frontend con React
18:00	**Tema 2**	Análisis inicial de una base de código React y detección de Code Smells
18:45	**Pausa**	15 minutos
19:00	**Ejercicio**	Diagnóstico de OrderDesk: hallazgos y matriz de deuda
19:30	**Corrección**	Solución comentada del ejercicio
19:45	**Cierre**	Proyecto final · Test de conceptos · Feedback individual

Mapa del curso: 10 sesiones, 30 horas

5 MIN

Objetivo final: poder cambiar UI, API, estado o reglas sin romper todo el sistema

1 Diagnóstico	2 Diseño	3 Refactor	4 Testing	5 Proyecto integrador
S1 · 10 nov Introducción y Code Smells	**S2 · 11 nov** SOLID, capas y drivers	**S3 · 12 nov** Features, Vertical Slice y dominio	**S4 · 16 nov** Casos de uso, Ports & Adapters	**S5 · 18 nov** Presentación, hooks, estado y routing
S6 · 19 nov Node, patrones, formularios y seguridad	**S7 · 23 nov** Rendimiento, refactoring y estrategia de testing	**S8 · 25 nov** Tests unitarios, componentes e integración	**S9 · 26 nov** E2E, CI/CD con IA, documentación y review	**S10 · 30 nov** Decisión, IA y proyecto final

OrderDesk: entorno y termómetro inicial

5 MIN

Un backoffice de pedidos en React, TypeScript y Node, con problemas sembrados a propósito

TERMINAL DE VISUAL STUDIO CODE

```
$ git clone https://github.com/joancifo/
    imagina-clean-react-s1.git
$ cd imagina-clean-react-s1
$ code .
$ npm install
$ npm run dev

# Web: http://localhost:5173 (usuario: Pau, manager)
# API: http://localhost:3001
```

Reglas de funcionamiento

- El chat es tu mando: votaciones y dudas
- Cámara y micrófono abiertos en los debates
- Rama propia: s1/<tu-nombre>
- Ranking: las dinámicas y los tests suman puntos

DINÁMICA · TERMÓMETRO · CHAT

1. Para cambiar una regla de negocio, ¿cuántos archivos tocas normalmente?

A) 1 B) 2 o 3 C) 4 o más D) No lo sé

2. ¿Dónde vive hoy la mayor parte de esa lógica?

A) Dominio B) Hooks C) Componentes D) Backend

3. Completa la frase:

«Lo que más me frena al cambiar código frontend es…»

AVISO Código, datos e historial de commits ficticios. No lo despliegues ni reutilices sus patrones: están ahí para detectarlos.

01

Introducción a Clean Architecture aplicada a frontend con React

Tema 1 · 17:15 – 18:00

Distinguir arquitectura de organización · Regla de dependencia · Coste y criterio de uso

Imagina

Arquitectura no es carpetas, estilo ni preferencia

5 MIN

Cuatro discusiones distintas que los equipos suelen mezclar

1 Arquitectura

Límites y dirección de las dependencias. Determina el coste de cambio.

Ejemplo

cancelOrder() no importa axios: depende de un contrato OrderGateway.

2 Carpetas

Dónde se ubican los archivos. Orienta, pero no separa por sí sola.

Ejemplo

Mover hooks/ a features/orders/hooks/ con los mismos imports.

3 Estilo de código

Cómo se escribe. Se automatiza con Prettier y ESLint.

Ejemplo

Comillas simples o dobles; punto y coma; longitud de línea.

4 Preferencia

Elección entre opciones equivalentes en este contexto.

Ejemplo

Zustand o Redux para un estado de interfaz bien aislado.

CRITERIO ¿Cambia el coste de modificar una pieza sin romper otra? Si la respuesta es sí, la discusión es de arquitectura.

¿Arquitectura, carpetas, estilo o preferencia?

5 MIN

Vota en el chat con una letra por afirmación: 1A, 2C…

DINÁMICA · VOTACIÓN · 1 PUNTO POR ACIERTO

1. Un componente importa axios para aprobar un pedido
2. Mover hooks/ a features/orders/hooks/ sin cambiar sus imports internos
3. Usar comillas simples o dobles
4. Los casos de uso no pueden importar React
5. Prefiero Zustand a Redux para el estado de la interfaz
6. Nombrar los archivos OrderRow.tsx u order-row.tsx

Opciones

A · Arquitectura

B · Carpetas

C · Estilo

D · Preferencia

DESPUÉS Las correcciones se razonan con el criterio de la diapositiva anterior: ¿qué cuesta cambiarlo?

Funciona… y está mal diseñada

5 MIN

La aplicación arranca, los 9 tests pasan y el usuario puede trabajar. Cambiarla es otra cuestión

UNA FUNCIÓN · SIETE RAZONES PARA CAMBIAR

```
// src/pages/OrdersPage.tsx:75 (resumido)
const handleApprove = async (order: Order) => {
  // Autorización
  if (user.usr_role !== 'admin' && ...) alert(...);
  // Reglas de negocio
  if ((order.total ?? 0) > 10000 && ...) alert(...);
  setSaving(true);                  // Estado de React
  // HTTP: fetch aquí, axios en services/api.ts
  const res = await fetch(`/api/orders/${id}/approve`);
  updateOrder(await res.json(), order.total);
  trackEvent('order_approved', { ... }); // Analítica
  setToast('Pedido aprobado');      // Interfaz
  // Navegación decidida por importe
  if ((order.total ?? 0) > 5000) navigate('/orders/' + id);
};
```

1 Componentes gigantes

OrdersPage: 210 líneas y complejidad 16

2 Negocio en la interfaz

Límites, descuentos y permisos en páginas

3 Hooks acoplados

useOrders: red, store, analítica y URL

4 Servicios mezclados

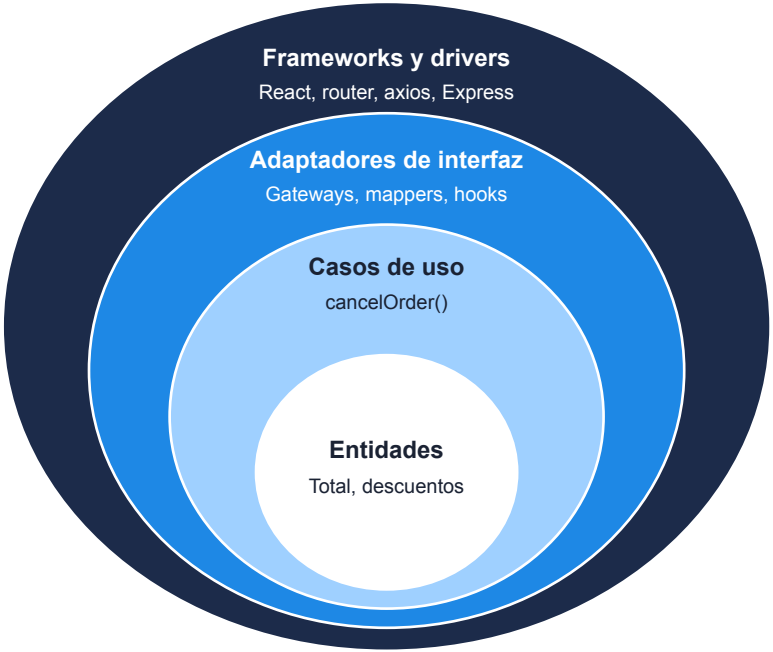
services/api.ts: HTTP y alert()

PRUEBA Ejecuta npm test: 9 tests en verde. El comportamiento actual no dice nada sobre el coste del próximo cambio.

La regla de dependencia

5 MIN

Robert C. Martin, «The Clean Architecture» (2012)



«Source code dependencies can only point inwards.»

Las dependencias de código fuente solo pueden apuntar hacia dentro.

PRUEBA RÁPIDA

```
// src/domain/order.ts
import axios from 'axios';          // NO: dominio -> detalle
// src/infrastructure/http-order-gateway.ts
import type { OrderGateway } from '../domain';  // OK
```

Pregunta para el chat

Un archivo de dominio importa formatCurrency de shared/utils. ¿Rompe la regla? Pista: revisa qué importa utils.ts.

MATIZ Dependencia de código no es dirección de la llamada: el caso de uso llama al adaptador, pero solo conoce su interfaz.

Qué aporta separar: el mapa aplicado a OrderDesk

5 MIN

Dominio, casos de uso, infraestructura, presentación y composición

Responsabilidad	Hoy en OrderDesk	Destino posible	Qué se gana
Dominio	Total y permisos en OrdersPage, utils, detalle y panel	order-rules.ts: funciones puras	Tests en milisegundos y una sola versión de cada regla
Casos de uso	handleApprove y handleCancel dentro de la página	approveOrder() y cancelOrder()	Intención legible y probable sin red
Infraestructura	axios en api.ts, fetch en la página, localStorage disperso	HttpOrderGateway, storage y analytics adapters	Cambiar proveedor afecta a un adaptador
Presentación	OrdersPage: filtros, reglas, API y JSX en una función	Componentes y hooks adaptadores	Cambios visuales sin riesgo para las reglas
Composición	No existe: cada módulo importa lo que necesita	Composition Root en el arranque	Sustituir implementaciones reales por fakes en tests

PREGUNTA ¿A qué fila pertenece el umbral de 3000 € que marca un pedido como urgente? ¿Y el color que lo resalta?

Demo: una regla sin React, navegador ni red

5 MIN

Abre examples/tema1/ y compara ambas versiones. Ejecuta el comando en tu terminal

ANTES · HAY QUE MONTAR LA PÁGINA

```
// src/pages/OrderDetailPage.tsx:21
const [total, setTotal] = useState(0);
useEffect(() => {
  if (!order) return;
  const sub = order.ln_items.reduce(
    (acc, l) => acc + l.qty * l.unit_prc, 0);
  const pct = order.cust_tier === 'gold' ? 0.1
    : order.cust_tier === 'silver' ? 0.05 : 0;
  setTotal(sub * (1 - pct) * 1.21);
}, [order]);
```

DESPUÉS · SIN IMPORTS NI FRAMEWORK

```
// examples/tema1/order-rules.ts
const DISCOUNT_BY_TIER = {
  gold: 0.1, silver: 0.05, standard: 0 };
const VAT = 0.21;

export function calculateOrderTotal(
  lines: OrderLine[], tier: CustomerTier) {
  const subtotal = lines.reduce((sum, line) =>
    sum + line.quantity * line.unitPrice, 0);
  return subtotal * (1 - DISCOUNT_BY_TIER[tier])
    * (1 + VAT);
}
```

```
$ npm run demo:tema1
 ✓ examples/tema1/cancel-order.test.ts (2 tests)
 ✓ examples/tema1/order-rules.test.ts (3 tests)     Tests  5 passed (5)
```

MATIZ Extraer una función no es hacer Clean Architecture: es el primer paso, y el de mejor relación entre coste y beneficio.

¿Cuándo aporta valor y cuándo es sobrediseño?

5 MIN

La arquitectura se paga por adelantado y se amortiza con cada cambio

Mantenibilidad | **Testabilidad** | **Onboarding** | **Velocidad de cambio** | **Coste de evolución**

Suele compensar

- Reglas de negocio complejas o con impacto económico
- Producto con años de vida prevista
- Varios equipos trabajando sobre la misma base
- Varias fuentes de datos o tecnologías con probabilidad de cambio
- Mucho coste por regresiones o falta de tests

Suele ser sobrediseño

- Prototipo para validar una idea
- Interfaz CRUD sin reglas de negocio propias
- Vida útil corta o producto con final previsto
- Interfaces con una sola implementación que nadie necesita sustituir ni probar
- Capas que solo reenvían llamadas

DINÁMICA Puntúa de 1 a 5 cuánto compensaría en tu proyecto actual, con un motivo. Un 1 bien argumentado es una respuesta válida.

El papel de Node.js en una arquitectura frontend

5 MIN

¿Forma parte del producto o solo del pipeline? La pregunta cambia las decisiones

Tooling

Pipeline

Vite, TypeScript, ESLint y npm se ejecutan sobre Node.

Scripts

Pipeline

scripts/map-imports.mjs analiza imports sin dependencias.

Testing

Pipeline

Vitest ejecuta reglas puras sin navegador.

BFF

Producto

Agrega APIs para una interfaz concreta. Sesión 6.

SSR

Producto

Renderiza React en servidor: sin window en código compartido.

```
// server/index.mjs: la API confía en la interfaz (sin permisos ni validación de estado)
app.post('/api/orders/:id/:action(approve|cancel|ship)', (req, res) => { ... });
// Para el dominio, Express es un driver, igual que React.
```

VOTACIÓN En tu proyecto, Node es: A) solo tooling · B) también BFF o SSR · C) también el backend principal.

Imagina

GitHub Copilot como apoyo arquitectónico

5 MIN

Analiza, compara y acelera. La decisión la comprende y la valida quien integra el cambio

Se puede delegar

- Explorar y resumir código
- Detectar candidatos a smell
- Comparar alternativas
- Generar borradores y tests

No se delega

- Decidir los límites
- Aceptar sin entender
- Integrar sin revisar

PROMPT P1 · CHAT DE COPILOT EN MODO ASK (Ctrl+Alt+I · ^⌘I)

```
#codebase Elabora una tabla de los módulos de src/
con: ruta, responsabilidad principal, otras
responsabilidades que asume y dependencias externas
(React, router, HTTP, localStorage).
Cita evidencia ruta:línea.
No propongas cambios todavía.
```

Tres comprobaciones de cada respuesta

- ¿La línea citada existe y dice lo que se afirma?
- ¿Una herramienta o un test puede confirmarlo?
- ¿La propuesta añade una abstracción que el problema no necesita?

DOCS

#codebase pide una búsqueda de código relevante, pero no adjunta todo el repositorio. Las instrucciones guían, no garantizan.

02

Análisis inicial de una base de código React y detección de Code Smells

Tema 2 · 18:00 – 18:45

Inventariar · Medir · Localizar reglas · Revisar tests · Priorizar

Seis preguntas, seis evidencias

5 MIN

Un smell es un indicio que hay que comprobar, no un veredicto

Pregunta	Comando	Qué aporta
¿Qué hay y cómo se conecta?	`npm run map`	Tamaño, imports salientes y entrantes
¿Hay dependencias circulares?	`npm run cycles`	Ciclos entre módulos (madge)
¿Dónde se concentra la complejidad?	`npm run smells`	Complejidad, tamaño y hooks (ESLint)
¿Qué dependencias sobran?	`npm run deps`	Paquetes sin usar (knip)
¿Qué protegen los tests?	`npm run coverage`	Código ejecutado, más lectura de los tests
¿Qué cambia con más frecuencia?	`npm run hotspots`	Commits por archivo (Git)

«A code smell is a surface indication that usually corresponds to a deeper problem in the system.» Martin Fowler

LÍMITE Ninguna herramienta detecta una regla contradictoria. Te dicen dónde mirar: el diagnóstico exige leer código y usar la aplicación.

Estructura, imports y dependencias circulares

5 MIN

Salida real de npm run map y npm run cycles sobre OrderDesk

INVENTARIO (FILAS PRINCIPALES)

```
$ npm run map
modulo                          lineas  salientes  entrantes
src/pages/OrdersPage.tsx           210          6          1
src/pages/OrderDetailPage.tsx       56          5          1
src/components/OrderRow.tsx         40          3          1
src/pages/DashboardPage.tsx         39          4          1
src/store/appStore.tsx              37          1          4
src/services/api.ts                 27          2          6
src/shared/utils.ts                 26          2          7
src/shared/types.ts                 20          0          8
```

MADGE

```
$ npm run cycles
Found 1 circular dependency!

1) services/api.ts >
   shared/utils.ts
```

Lectura del ciclo

api usa getErrorMessage; utils usa sendLog.

Organización por tipo

La carpeta no revela el negocio: no aparece «pedidos».

Punto de concentración

OrdersPage: 210 líneas y 6 imports internos.

Alto impacto de cambio

types.ts, utils.ts y api.ts: 8, 7 y 6 entrantes.

AUTOMATIZA madge termina con código de salida 1 si hay ciclos: se puede convertir en un control de CI.

Componentes: tamaño, props y boolean flags

5 MIN

Salida real de npm run smells y el contrato de OrderRow

ESLINT (SALIDA ABREVIADA)

```
$ npm run smells
OrderRow.tsx
  24:16 complexity of 15. Maximum allowed is 10
OrdersPage.tsx
  11:16 'OrdersPage' has too many lines (182).
        Maximum allowed is 80
  11:16 'OrdersPage' has a complexity of 16.
  75:46 Async arrow function: complexity of 14.
  29:6  useEffect missing dependency 'setFilters'
9 problems (0 errors, 9 warnings)
```

CONTRATO DEL COMPONENTE

```
// src/components/OrderRow.tsx:5
type Props = {
  order: Order;
  showTotal: boolean; showDate: boolean;
  showOwner: boolean; isCompact: boolean;
  isAdmin: boolean; isManager: boolean;
  isHighlighted: boolean; disabled: boolean;
  canApprove: boolean; canCancel: boolean;
  ownerName?: string;
  onApprove: ...; onCancel: ...; onOpen: ...;
};  // 15 props, 10 booleanas: 1024 combinaciones
```

JSX con negocio

OrdersPage.tsx:186: pendiente y más de 3000 € = urgente.

Regla repetida

La regla de urgente también está en OrdersPage.tsx:53 y mappers.ts:8.

Rol en la vista

isAdmin e isManager llegan a una fila de tabla.

PREGUNTA En lugar de isAdmin, isManager, canApprove y canCancel, ¿qué debería recibir OrderRow?

useEffect como pegamento

5 MIN

React.dev: «Effects are an escape hatch». Si un valor se puede calcular, no necesita un efecto

ESTADO DERIVADO SINCRONIZADO A MANO

```
// OrdersPage.tsx:37 y :49 (resumido)
const [filteredOrders, setFilteredOrders] =
  useState<Order[]>([]);
const [pendingCount, setPendingCount] = useState(0);

useEffect(() => {
  setFilteredOrders(applyFilters(orders, filters));
}, [orders, filters]);

useEffect(() => {
  setPendingCount(countPending(filteredOrders));
}, [filteredOrders]);
```

CALCULADO DURANTE EL RENDER

```
// Sin estado adicional ni efectos:
const filteredOrders = applyFilters(orders, filters);
const pendingCount = countPending(filteredOrders);

// Solo si el cálculo es realmente costoso:
const sorted = useMemo(
  () => expensiveSort(filteredOrders),
  [filteredOrders],
);
```

Efecto	Qué hace	Valoración
1, 2 y 5	localStorage y temporizador	Sistema externo: legítimo, aunque el acceso debería aislarse
3 y 4	Lista filtrada y contadores	**Estado derivado: se puede eliminar**

OCULTO useOrders() no recibe nada, pero depende del store, la red, analytics y window.location. useCurrentUser escribe en localStorage.

La misma regla, tres respuestas

5 MIN

Dónde viven las reglas de negocio de OrderDesk: compruébalo tú en la aplicación

TRES VERSIONES DE «¿PUEDE CANCELAR UN MANAGER?»

```
// Lista · OrdersPage.tsx:66
if (role === 'manager')
  return o.ord_st === 'pending' && total < 5000;
// Detalle · OrderDetailPage.tsx:31
role === 'manager' && order.ord_st !== 'shipped'
// Panel · DashboardPage.tsx:26
role === 'manager' && r.status === 'pending'
  && r.total <= 6000  // total sin descuento
```

Pantalla	¿Cancelar P-1003?
Lista	No
Detalle	Sí
Panel	No (excluido)

DINÁMICA · ¿Cuál es correcta?

A) Lista B) Detalle C) Panel D) Otra

También en el total

- P-1002: 248,29 € en el detalle y 261,36 € en el panel
- Facturación: 769,56 € en el panel; 756,49 € con los totales de la lista
- Causa: DashboardPage.tsx:18 no aplica el descuento

Qué demuestra

- Ninguna herramienta ni test lo ha detectado
- El código no puede responder: hay que preguntar a negocio
- Shotgun Surgery: una corrección exige tocar varios sitios

LÍMITE La autorización visual no protege: server/index.mjs no comprueba permisos. La autorización real corresponde al backend (Sesión 6).

Smells arquitectónicos y acoplamientos

5 MIN

No se ven en un solo archivo: aparecen al mirar cómo se relacionan los módulos

Smell	Evidencia en OrderDesk	Pregunta de detección
shared sin criterio	shared/utils.ts: moneda, fechas, total, errores, analítica	¿Quién es el dueño?
Servicio genérico	api.ts:8 doRequest; api.ts:18 alert en la capa HTTP	¿Expresa una intención?
Dos clientes HTTP	axios en api.ts; fetch en OrdersPage.tsx:98	¿Hay una sola política de errores?
Mappers duplicados	mappers.ts:4 y DashboardPage.tsx:15	¿Cuál es la versión correcta?
Tipos sin propietario	types.ts: Order mezcla API, negocio y rowColor	¿Para quién existe este campo?
Modelo de backend en UI	ord_st, cust_nm, ln_items…: 33 líneas en 7 archivos	¿Qué pasa si se renombra?
Dependencias innecesarias	classnames sin usar; moment y lodash para una sola función cada una	¿Compensa lo que aporta?

PRUEBA Ctrl+Shift+F (⇧⌘F en macOS) y busca ord_st: cuenta cuántos archivos cambiarían si el backend lo renombrara.

¿Qué protegen realmente los tests?

5 MIN

Salida real de npm run coverage y experimento con OrdersPage.test.tsx

ACOPLADO A CSS, DOM Y HTML COMPLETO

```
// src/pages/OrdersPage.test.tsx:41
container.querySelector('.order-row--urgent')
container.querySelector(
  'tbody tr td:nth-child(6)')?.className
container.querySelectorAll('.order-row button')
  .length  // toBe(2)
// :49
expect(table.innerHTML).toMatchSnapshot();
```

COBERTURA (ABREVIADA)

```
$ npm run coverage
File              % Stmts % Funcs Uncovered
All files           53.56   34.21
OrdersPage.tsx      64.24   15.38 76-111 y
                                  114-127
OrderDetail...          0       0 1-56
DashboardPage           0       0 1-39
```

EXPERIMENTO · EN TU TERMINAL

1. Borra OrdersPage.tsx:80-95 (permisos y validaciones de aprobar)
2. En la línea 66, cambia el límite de 5000 por 1
3. npm test 4. git checkout src/pages/OrdersPage.tsx

9 / 9

tests en verde tras eliminar los permisos

CONCLUSIÓN La cobertura mide qué líneas se ejecutan, no qué comportamiento se comprueba: fallan sin motivo y no fallan cuando deberían.

Priorizar: hotspots y matriz de deuda

5 MIN

No toda deuda merece pagarse ahora: importa lo problemático que además cambia con frecuencia

CAMBIO FRECUENTE (HISTORIAL SIMULADO)

```
$ npm run hotspots
archivo                          commits
src/pages/OrdersPage.tsx              15
src/pages/OrderDetailPage.tsx          4
src/services/api.ts                    3
src/shared/utils.ts                    3
```

Hallazgo	Impacto	Riesgo	Facilidad	Prior.
Total en 3 sitios	5	4	4	36

Prioridad = (Impacto + Riesgo) × Facilidad

Valores de 1 a 5. Convención del curso para ordenar la discusión: no es un estándar ni sustituye al juicio.

Problema	Criterio automático para evitar la regresión
Dependencias circulares	madge --circular en CI: falla si aparece un ciclo
Componentes y complejidad	ESLint: complexity y max-lines como error en código nuevo
Dependencias sin usar	knip en CI: sin --no-exit-code

FUENTES Hotspots: A. Tornhill, Your Code as a Crime Scene. Deuda prudente o imprudente: M. Fowler, Technical Debt Quadrant.

Copilot y análisis estático: cómo se combinan

5 MIN

Las herramientas aportan datos, Copilot acelera la lectura y la persona comprueba y decide

1 Medir

Scripts, madge, ESLint, knip y cobertura

2 Preguntar

Copilot en modo Ask con evidencia ruta:línea

3 Comprobar

Abrir la línea y usar la aplicación

4 Proponer

Un cambio pequeño y reversible, con tests

PROMPT P2 · MODO ASK · prompts/sesion-1.md

```
#codebase Localiza todas las reglas de negocio
de pedidos: total, descuentos, permisos y
transiciones de estado. Para cada regla, lista dónde
aparece e indica si las versiones coinciden.
Cita ruta:línea y separa hechos de hipótesis.
```

Rechaza la propuesta si…

- Reescribe en lugar de evolucionar
- Añade una librería sin justificarla
- Crea una abstracción para un solo caso
- Cita líneas que no existen
- Sus tests replican la implementación

REGISTRO La salida del chat varía entre ejecuciones. Documenta solo lo que hayas verificado.

15

Pausa ·15 minutos

Volvemos a las 19:00 con el ejercicio
Para pensar: si solo pudieras hacer un Pull Request en OrderDesk, ¿cuál sería?

Parte A: hallazgos con evidencia

15 MIN

Individual · 19:00 – 19:15 · Enunciado completo en docs/ejercicio-s1.md

1 · DATOS

```
$ git checkout -b s1/<tu-nombre>
$ npm run map
$ npm run cycles
$ npm run smells
$ npm run deps
$ npm run coverage
$ npm run hotspots
```

2 Comprobar en la aplicación

Con npm run dev, como manager, abre P-1002 y P-1003 en lista, detalle y panel. Anota las diferencias.

3 Registrar al menos 6 hallazgos

Categorías: regla de negocio · acoplamiento · componente · hook/efecto · estructura · test · dependencia.

FORMATO

```
| # | Hallazgo | Categoría |
| Evidencia ruta:línea |
| Por qué encarece el cambio |

Archivo: docs/diagnostico-<tu-nombre>.md
```

CALIDAD Sin evidencia ruta:línea no cuenta. Copilot (P1 a P3) está permitido si abres y compruebas cada línea citada.

Parte B: matriz de deuda y primer Pull Request

15 MIN

Individual · 19:15 – 19:30 · Añádela a tu diagnostico-<tu-nombre>.md

PLANTILLA · TUS 5 HALLAZGOS PRINCIPALES

```
| # | Impacto | Riesgo | Facilidad | Prioridad | Primer paso pequeño | Criterio automático |
| H1 |        |        |           |           |                     |                     |

Prioridad = (Impacto + Riesgo) × Facilidad · Valores de 1 a 5 · Facilidad 5 = fácil y seguro
```

1 Puntúa

Impacto, riesgo y facilidad, con un motivo en caso de duda.

2 Decide

Primer paso que quepa en un Pull Request y criterio automático contra la regresión.

3 Entrega

«Mi primer Pull Request sería … porque …». Haz commit y push de tu rama y comparte el enlace en el chat.

CRITERIO No hay una matriz única. Se valora la coherencia entre evidencia, puntuación y primer paso.

Solución: hallazgos de referencia

10 MIN

Compara con los tuyos. Cualquier hallazgo adicional con evidencia también es válido

#	Hallazgo	Categoría	Evidencia
H1	Total en 3 sitios; el panel omite el descuento	Regla	utils.ts:13 · OrderDetailPage.tsx:23 · DashboardPage.tsx:18
H2	Cancelación con 3 versiones contradictorias	Regla	OrdersPage.tsx:64 · OrderDetailPage.tsx:31 · DashboardPage.tsx:24
H3	handleApprove: 7 responsabilidades y sin ejecutar en tests	Componente	OrdersPage.tsx:75; cobertura 76-111
H4	Dependencia circular	Estructura	api.ts:3 ↔ utils.ts:2; npm run cycles
H5	Modelo de backend en la interfaz	Acoplamiento	types.ts:2; 33 líneas en 7 archivos
H6	Estado derivado sincronizado con efectos	Hook/efecto	OrdersPage.tsx:37 y :49 · OrderDetailPage.tsx:21
H7	15 props, 10 booleanas y roles en una fila	Componente	OrderRow.tsx:5
H8	Dependencias sin uso o con poco uso	Dependencia	knip: classnames; utils.ts:1; OrdersPage.tsx:3

OTROS También válidos: useCurrentUser.ts:9 · api.ts:18 (alert) · appStore.tsx (store único) · umbral de urgente repetido · server sin permisos.

Solución: matriz de referencia y primer PR

5 MIN

Una priorización razonada: la tuya puede ser distinta si la argumentas

Hallazgo	Impacto	Riesgo	Facil.	Prior.	Criterio automático
H1 Total en 3 sitios	5	4	4	**36**	Tests de calculateOrderTotal por nivel de cliente
H2 Cancelación	5	5	3	**30**	Tabla de tests sobre una única regla canCancel
H4 Ciclo api ↔ utils	2	3	5	**25**	madge --circular en CI
H6 Efectos derivados	3	2	5	**25**	Revisión y test de filtros (la regla de lint no lo cubre todo)
H3 handleApprove	4	4	2	**16**	Characterization tests antes de tocarlo
H5 Backend en UI	4	3	2	**14**	Regla de importación: la UI no usa DTO

Primer Pull Request de referencia: H1

Tests, función pura calculateOrderTotal y sustitución de las copias de utils y detalle, sin cambiar el comportamiento. En un commit aparte, corregir el panel: 769,56 € pasa a 756,49 €, previa confirmación de negocio.

CLAVE Un refactor conserva el comportamiento; corregir el panel lo cambia. Por eso van en commits separados y se explican en el PR.

Proyecto final evolutivo

5 MIN

Transformar la feature Pedidos de OrderDesk, de forma incremental, sesión tras sesión

S1 Diagnóstico y matriz	**S2** Reglas de importación y capas	**S3** Estructura y reglas puras	**S4** Casos de uso, puertos y Composition Root	**S5** Hooks, view models, estado y routing
S6 Node, formularios y seguridad	**S7** Medición, refactor y estrategia de tests	**S8** Tests unitarios, componentes e integración	**S9** E2E, CI/CD y ADR	**S10** Entorno de IA y presentación

Evidencia | **Trade-offs** | **Incrementalidad** | **Protección** | **IA responsable**

Criterios de evaluación. Decidir no aplicar un patrón, con un motivo, también cuenta como buena decisión.

`Detalle en docs/proyecto-final.md`

ALCANCE Refactor incremental, no reescritura: el comportamiento solo cambia cuando se decide y se documenta.

Test de conceptos: 5 preguntas, 30 segundos

5 MIN

Individual · Responde en el chat al final: 1A, 2C… · 1 punto por acierto

1. En Clean Architecture, las dependencias de código fuente…

A) apuntan hacia dentro B) apuntan hacia fuera C) van en ambos sentidos

2. Mover hooks/ a features/orders/hooks/ sin cambiar imports es…

A) cambio de arquitectura B) organización de carpetas C) estilo

3. Un valor calculable a partir de props o estado se debe…

A) calcular durante el render B) sincronizar con useEffect C) guardar en un store

4. Un code smell es…

A) un bug confirmado B) un error de ESLint C) un indicio que suele indicar un problema más profundo

5. Un 64 % de líneas cubiertas, con los handlers sin ejecutar, significa que…

A) las reglas están protegidas B) hay que borrar los tests C) esas reglas no están protegidas

RANKING Los puntos se suman a las dinámicas de hoy. El ranking se anuncia al inicio de la Sesión 2.

Feedback individual y próxima sesión

5 MIN

Miércoles 11 de noviembre · 17:00 – 20:00 · Acoplamiento, cohesión, SOLID y capas

FEEDBACK · MENSAJE PRIVADO EN EL CHAT

1. La idea que te llevas de hoy

2. El concepto que quieres repasar mañana

Tu rama recibirá un comentario individual.

Tres ideas de hoy

- Funcionar no significa estar bien diseñada
- Las dependencias apuntan hacia dentro
- Un diagnóstico se apoya en evidencias

Antes de la Sesión 2

- Completa docs/diagnostico-<tu-nombre>.md
- Haz push de tu rama s1/<tu-nombre>
- Deja funcionando npm run dev y npm test

Sesión 2 · 11 de noviembre

- Repaso
- Acoplamiento, cohesión y SOLID en React y TypeScript
- Capas, responsabilidades, frameworks y drivers

CONTINUIDAD Tu matriz es el punto de partida de mañana: qué principio incumple cada hallazgo y qué regla de importación lo evita.

Fuentes de la sesión

ANEXO

Material de consulta. No se expone en clase

Fuente	Ubicación
R. C. Martin · The Clean Architecture (2012)	blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html
React · You Might Not Need an Effect	react.dev/learn/you-might-not-need-an-effect
M. Fowler · Code Smell	martinfowler.com/bliki/CodeSmell.html
M. Fowler · Technical Debt Quadrant	martinfowler.com/bliki/TechnicalDebtQuadrant.html
Refactoring Guru · Code Smells	refactoring.guru/refactoring/smells
A. Tornhill · Your Code as a Crime Scene	Libro (The Pragmatic Bookshelf)
VS Code · GitHub Copilot: funciones	code.visualstudio.com/docs/copilot/reference/copilot-vscode-features
VS Code · Contexto en el chat	code.visualstudio.com/docs/copilot/chat/copilot-chat-context
VS Code · Instrucciones personalizadas	code.visualstudio.com/docs/copilot/customization/custom-instructions
knip · Unused dependencies	knip.dev/typescript/unused-dependencies
madge	github.com/pahen/madge


Imagina